

Documento de Especificaciones Técnicas

MICROSERVICIO DE OPTIMIZACIÓN DE IMÁGENES & TRANSCODIFICACIÓN WEBP

Proyecto:	Media Optimizer Edge Layer	Fecha:	11 de Junio, 2026
Estado:	Listo para Implementación	Autor:	Arquitectura de Software
Entorno Base:	Docker / VPS Privado	Enrutamiento:	Traefik Reverse Proxy

1. Resumen Ejecutivo y Objetivos

El objetivo de este documento es definir la arquitectura, endpoints y pipeline de procesamiento para un microservicio autónomo de optimización de imágenes. El sistema interceptará o recibirá solicitudes de recursos multimedia primitivos (PNG, JPEG, TIFF) para devolver versiones optimizadas dinámicamente y codificadas nativamente en formato **WebP**.

Metas principales:

- Reducir el peso de los recursos gráficos en un promedio del 60% al 85% sin pérdida de calidad perceptible para el ojo humano.
- Proveer compatibilidad total con arquitecturas web desacopladas (headless webs).
- Garantizar un tiempo de respuesta de procesamiento en frío inferior a 250ms, y sub-10ms para recursos cacheados.

2. Arquitectura del Sistema

El servicio se estructurará bajo un modelo de procesamiento bajo demanda (on-the-fly) o mediante pre-generación por API, actuando como una capa interna adyacente a la distribución de contenido. Se opta por una solución basada en **Node.js** utilizando la librería de bajo nivel **Sharp** debido a su rendimiento de memoria superior (basado en la biblioteca C `libvips`) frente a alternativas como ImageMagick.

Nota de Rendimiento
Sharp procesa imágenes de forma multihilo de manera nativa, lo que maximiza el aprovechamiento de los núcleos de CPU del VPS asignado, evitando el bloqueo del bucle de eventos principal de Node.js.

3. Especificación de Endpoints (API REST)

El microservicio expondrá una interfaz HTTP simplificada para la manipulación y obtención de recursos optimizados.

3.1. Endpoint de Conversión Directa (POST)

URL: /api/v1/optimize

Maneja cargas de archivos binarios directos para flujos de trabajo de administración o flujos automatizados (n8n, CMS corporativos, etc.).

Parámetro (Form-Data)	Tipo	Requerido	Descripción
image	Binary File	Sí	Archivo original (PNG, JPG, JPEG, WEBP). Máximo 15MB.
quality	Integer	No	Nivel de compresión WebP (1-100). Valor por defecto: 80.
width	Integer	No	Ancho objetivo para redimensionamiento en píxeles. Mantiene aspecto.

3.2. Estructura de Respuesta (Sucess 200 OK)

Retorna un flujo binario con el tipo MIME correspondiente a image/webp junto con cabeceras de control de caché personalizadas.

Headers:

Content-Type: image/webp

Cache-Control: public, max-age=31536000, immutable

X-Optimizer-Saving: 74.3% (Ejemplo de métrica de ahorro)

4. Pipeline de Procesamiento del Motor (Sharp Logic)

La lógica del núcleo debe encapsular el manejo de buffers en memoria para evitar operaciones de escritura en disco innecesarias que penalicen la velocidad de E/S. A continuación se detalla la secuencia algorítmica:

```

const sharp = require('sharp');

async function processImage(inputBuffer, options = {}) {
  const { quality = 80, width } = options;

  let pipeline = sharp(inputBuffer);

  if (width) {
    pipeline = pipeline.resize({
      width: parseInt(width),
      withoutEnlargement: true // Evita pixelación si la imagen es menor
    });
  }

  return await pipeline
    .webp({
      quality: parseInt(quality),
      effort: 4, // Balance óptimo entre velocidad de CPU y compresión
      lossless: false
    })
    .toBuffer();
}

```

5. Infraestructura y Configuración de Despliegue

El microservicio se desplegará de forma aislada e inmutable mediante un contenedor Docker, integrado dentro de la arquitectura de red gestionada por Dokploy en el VPS, utilizando Traefik como Proxy Inverso para el manejo de certificados TLS y balanceo.

5.1. Dockerfile de Producción Optimizada

Se utiliza una imagen base ligera de Node junto con la instalación limpia de dependencias de producción:

```
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

FROM node:18-alpine AS runner
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY . .

EXPOSE 3000
ENV NODE_ENV=production
CMD ["node", "server.js"]
```

5.2. Configuración de Etiquetas Traefik (para Dokploy/Compose)

Para asegurar que las solicitudes se enruten correctamente y que el tráfico estático sea protegido por HTTPS de forma automática, se deben incluir las siguientes directivas de red:

- `traefik.http.routers.img-opt.rule=Host(`cdn-optimizer.tudominio.com`)`
- `traefik.http.routers.img-opt.entrypoints=websecure`
- `traefik.http.routers.img-opt.tls.certresolver=letsencrypt`

6. Consideraciones de Seguridad y Escalabilidad

1. **Protección contra ataques DDOS de descompresión:** Limitar el tamaño máximo del cuerpo de la solicitud HTTP (Body Size Limit) en la capa de Traefik y Node.js a un estricto máximo de 15MB por archivo para evitar el agotamiento de memoria RAM.
2. **Estrategia de Caché:** Al procesar imágenes estáticas de una URL de origen externa, se debe implementar una capa de almacenamiento en caché permanente (ej. Redis o un volumen local montado en Docker para almacenar en caché los resultados de Sharp correspondientes a firmas hash MD5 de los archivos de entrada).
3. **Manejo de Errores:** Si el archivo binario de entrada está corrupto o no es soportado, el servicio responderá con un código 422 `Unprocessable Entity` en formato JSON limpio, sin exponer stack-traces del sistema.